

Practice 06: Deep Learning and Deep Learning Agents

Intelligent Systems II

Mário Antunes

2026

Introduction

In this practice, we will explore the transition from classical machine learning to **Deep Learning (DL)**. We will use **Keras 3** with the **JAX** backend to build and train Neural Networks.

The focus of the second half of this guide is **Behavioral Cloning (Imitation Learning)**. Since we are not using Reinforcement Learning, we will train our agents to mimic “experts”—algorithmic solutions (like Minimax, BFS, or simple heuristics) that already know how to play the games.

Setup

Ensure you have the following packages installed:

```
pip install keras jax jaxlib numpy tqdm
```

Set the Keras backend to JAX in your environment:

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

Part 1: Foundational MLP

Your first task is to train a simple Multi-Layer Perceptron (MLP) to classify non-linearly separable data (concentric circles).

Task:

1. Generate a dataset using the provided `generate_data` function.
2. Build a Keras Sequential model with:
 - An Input layer for 2D coordinates.
 - At least two hidden Dense layers (e.g., 16 units) using `relu` activation.
 - A final Dense layer with 1 unit and `sigmoid` activation.
3. Compile the model with the `adam` optimizer and `binary_crossentropy` loss.
4. Train for 20 epochs and verify the accuracy.

Hint:

- Use `model.summary()` to inspect the number of parameters.
- For non-linear boundaries, deeper networks or more neurons are often better than shallow ones.

Skeleton:

```
import os
os.environ["KERAS_BACKEND"] = "jax"
import keras
import numpy as np
```

```
def generate_data(n=1000):
    X = np.random.randn(n, 2)
    y = (np.sum(X**2, axis=1) < 1).astype(int)
    return X, y
```

```
X, y = generate_data()
```

```
model = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    # Add your layers here
])
```

```
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["acc"])
model.fit(X, y, epochs=20, batch_size=32)
```

Part 2: Convolutional Neural Networks (CNN)

Images contain spatial patterns that MLPs struggle to capture efficiently. Use a CNN to recognize handwritten digits.

Task:

1. Load the MNIST dataset using `keras.datasets.mnist.load_data()`.
2. **Preprocess:**
 - Normalize pixel values to the $[0, 1]$ range.
 - Reshape the input to $(28, 28, 1)$ to account for the single color channel.
 - Convert labels to one-hot encoding using `keras.utils.to_categorical`.
3. **Architecture:**
 - Use Conv2D layers with 3×3 kernels.
 - Add MaxPooling2D to reduce spatial dimensionality.
 - Flatten the maps before passing them to the final Dense layer.
4. Train and evaluate the model's performance on the test set.

Hint:

- A common pattern is: Conv2D $\rightarrow$ MaxPool $\rightarrow$ Conv2D $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense.
 - The softmax activation in the final layer is essential for multi-class classification (10 digits).
-

Part 3: Behavioral Cloning - The Hot/Cold Agent

In Behavioral Cloning, we teach an agent to mimic an expert. Our expert for the Hot/Cold game is a **Binary Search** algorithm.

Task:

1. **Data Generation:** Implement a script that simulates games where an expert uses binary search. Save the `[low, high]` bounds as features and the mid point as the target action.
2. **Training:** Train an MLP to predict the normalized guess (0.0 to 1.0) given the normalized bounds.
3. **Inference:** Create a "Play" script where the agent receives feedback ("Higher" or "Lower") from the environment, updates its local bounds, and

asks the model for the next guess.

Hint:

- Normalize your inputs and outputs to $[0, 1]$ by dividing by 100. Neural networks converge much faster on normalized data.
 - The model should use a linear activation (or no activation) in the final layer for this regression task.
-

Part 4: Strategic Play - Tic-Tac-Toe

approximation of a recursive search (Minimax) using a Neural Network.

Task:

1. **Expert:** Use a Minimax algorithm to generate optimal moves for 5000 random board states.
2. **Representation:** Flatten the 3×3 board into a 9-element vector (1: Agent, -1 : Opponent, 0: Empty).
3. **Training:** Train a classifier to predict the index (0 — 8) of the best move.
4. **Validation:** Test the agent against a random player. Does it always block the opponent's winning moves?

Hint:

- Since you are always predicting the move for the current player, you can flip the board (multiply by -1) if it's the opponent's turn, so the model always sees itself as player 1.
 - Use CategoricalCrossentropy loss for the 9 possible move indices.
-

Part 5: Visual Navigation - Snake CNN

A pinnacle of this practice: training a CNN to play Snake using only a visual representation of the grid.

Task:

1. **Environment:** Implement a Snake simulation on a 10×10 grid.
2. **State:** Create a $10 \times 10 \times 3$ tensor:
 - Channel 0: Snake body.

- Channel 1: Snake head.
 - Channel 2: Food.
3. **Expert:** Use Breadth-First Search (BFS) to find the shortest path to the food at every step.
 4. **Training:** Train a CNN (similar to the MNIST one) to predict the direction [Up, Down, Left, Right].

Hint:

- The BFS expert should avoid the snake's own body.
 - Visual states are powerful because the CNN can learn spatial relationships (e.g., "The food is above a body segment").
 - If the snake crashes often, try generating more training data from "dangerous" states where the snake is close to a wall or its own tail.
-

Conclusion

By completing these exercises, you have moved from simple mathematical mappings (Part 1) to complex agents that "perceive" their world (Part 5) and act based on learned expertise.

Next Steps: * Try combining different architectures (e.g., a CNN for perception feeding into an MLP for decision-making). * Explore how the agent performs if the "Expert" is not perfect.